

LLD Notes — Phase 2

SOLID principles · DRY, KISS, YAGNI | Concepts, your doubts, your mistakes, interview answers

Colour key. Blue = a doubt you asked · Red = a mistake you made · Green = rule to remember · Amber = how to say it in an interview.

Why SOLID matters in the room. Interviewers rarely ask "define SOLID". They ask "how would you extend this?" or "what's wrong with this class?" — and the strong answer names the principle. "That violates Open/Closed, so I'd pull pricing into a strategy" sounds completely different from "I'd add another if".

You had already done four of the five by hand before naming them

Principle	Where you already did it
S — Single Responsibility	Exercise 11 — splitting BookingManager
O — Open/Closed	Exercises 5 & 6 — new channel / payment type = new class, nothing edited
L — Liskov Substitution	The FixedDepositAccount that refused withdrawals
I — Interface Segregation	Bonusable kept separate from Employee (exercise 7)
D — Dependency Inversion	Constructor injection of interfaces — every exercise from 5 onward

S — Single Responsibility Principle

Definition: a class should have only **one reason to change**. Sharper version: **a class should answer to one actor** — one group of people who would ask for it to change.

"One responsibility" is vague; "one reason to change" is testable.

```
class Employee {
    public double calculatePay() { ... } // Finance owns these rules
    public String reportHours() { ... } // HR owns this format
    public void save() { ... } // Infra/DBA owns this
}
```

It looks cohesive — everything is about an employee. But three teams would request changes.

The concrete danger. Suppose calculatePay() and reportHours() share a private helper regularHours(). Finance asks for overtime to be counted differently, someone edits the helper — and **HR's report silently changes too**, though HR never asked. SRP exists to stop one actor's change from breaking another actor's feature.

```
class Employee { /* data only */ }
class PayCalculator { double calculatePay(Employee e); } // Finance
class HoursReporter { String reportHours(Employee e); } // HR
class EmployeeRepository { void save(Employee e); } // Infra
```

Two tests you can use out loud

- Describe the class in one sentence without using "and".**
- List who would ask for it to change.** More than one team or role = more than one reason.

Counterweight — SRP is NOT "one method per class". If ParkingSpot has occupy(), release(), isAvailable(), leave them together: they change for the same reason (the rules of being a spot). **Things that change together belong together.** Sign you over-split: every change still touches three of your tiny classes at once.

Your mistake (exercise 12): you wrote "lots of processing steps" and named customers and lenders as actors. Customers **receive** invoices — they never request code changes. Name the internal actors:

Part of the method	Actor who would ask for a change
Discount switch	Sales / marketing — new tiers, campaign offers
18% tax	Finance / tax compliance — rate changes by law or region
TEXT / CSV formatting	Product / accounts — customers want PDF or HTML, extra column
Emailing	Infra / comms — switch provider, add SMS

Four actors in one method. Naming them this concretely is what makes an SRP answer sound strong.

O — Open/Closed Principle

Definition: software should be **open for extension, closed for modification** — add new behaviour by writing **new** code, without editing working, tested code.

```
// VIOLATION
class DiscountCalculator {
    public double discount(String customerType, double amount) {
        if (customerType.equals("REGULAR"))    return 0;
        else if (customerType.equals("GOLD"))    return amount * 0.10;
        else if (customerType.equals("PLATINUM")) return amount * 0.20;
        return 0;
    }
}
```

Every new type reopens this method. Three risks: you can break an existing branch, the class must be retested each time, and the method grows forever. Worse, such chains get **copied** into other places (invoice text, eligibility checks), so each new type means hunting every copy.

```
// FIX – polymorphism (this is exactly why Strategy exists)
interface DiscountPolicy { double discount(double amount); }

class NoDiscount implements DiscountPolicy { public double discount(double a){ return 0; } }
class PercentDiscount implements DiscountPolicy {
    private final double rate;
    public PercentDiscount(double rate) { this.rate = rate; }
    public double discount(double a) { return a * rate; }
}

class Checkout {
    private final DiscountPolicy policy;
    public Checkout(DiscountPolicy policy) { this.policy = policy; }
    public double finalAmount(double amount) { return amount - policy.discount(amount); }
}
```

Something still has to choose — and that's fine

A fair interviewer push-back: *somewhere* code still turns "GOLD" into a GoldDiscount . Yes — but that choice moves to **one place** (a map or factory) instead of being scattered through business logic:

```
Map<String, DiscountPolicy> policies = Map.of(
    "REGULAR", new NoDiscount(),
    "GOLD",    new PercentDiscount(0.10),
    "PLATINUM", new PercentDiscount(0.20),
    "STUDENT", new PercentDiscount(0.15)    // ← the ONE line you add for a new type
);
```

Notice: GOLD, PLATINUM and STUDENT are **the same behaviour with different data** → one parameterised class, not three classes. Recognising this prevents over-splitting.

How to spot OCP violations fast

- `if/else` or `switch` on a **type code** (String, enum, `instanceof`)
- The same branching logic **repeated** in several places
- A method you keep reopening every time requirements grow

Counterweight: you can't close against every possible change. Close against the **axis of change you actually expect**. Two cases that never change? A plain `if` is fine. An **enum with behaviour** is a lighter middle ground when variants are data-like and few — adding a constant is a one-file edit.

The stock question: "How would you add a new payment type / discount tier / notification channel?" **Strong answer:** "Write one new class implementing the interface, plus one registration line. No existing file changes." That *is* the OCP test, and passing it is the point of the whole design.

- Exercise 12 — what you got right and wrong.**
- ✓ Part C passed: new discount class + new formatter class, no edits to existing logic. That's the real proof of OCP.
 - ✓ Correct instinct to inject all collaborators into an orchestrator.
 - ✗ Actors named vaguely (see the SRP box above).
 - ✗ `calculateDiscount(); / formatInvoice();` — **no return types, no parameters**. In design work the **signature IS the design**: it defines what data flows between classes. Hiding it hid the hardest question.
 - ✗ Missed the **value object**. A formatter needs subtotal, discount, tax, total — four loose doubles are easy to mis-order. Introduce `Invoice` and the seam between calculation and presentation becomes clean.
 - ✗ PricingStrategy that "includes with-tax and without-tax" mixes **sales** and **finance** actors — the exact thing SRP separates.
 - ✗ Missed the registration line in Part C.

```
class Invoice {
    // the value object that flows between the halves
    private final double subtotal, discount, tax, total;
    public Invoice(double subtotal, double discount, double tax) {
        this.subtotal = subtotal; this.discount = discount;
        this.tax = tax; this.total = subtotal - discount + tax;
    }
    // getters
}

interface InvoiceFormatter { String format(Invoice invoice); }
```

On the tax question I left you: extract a `TaxPolicy` if the rate varies by region/product or changes by law (finance owns it, separate axis). Keep it a named constant `private static final double GST = 0.18;` if there's one country and one rate — extracting then is YAGNI. **Either is defensible if you say why.** What's never right is folding it into a class owned by a different actor.

L — Liskov Substitution Principle

Definition: if S is a subtype of T, you must be able to use an S anywhere a T is expected without the program behaving incorrectly. Plainly: **a subclass must keep every promise its parent makes**. Code written against the parent should never need to know which subclass it got.

Most inheritance bugs in LLD are Liskov violations. It is also the principle most often asked **by name**, and the one people explain worst.

```
class BankAccount { public void withdraw(double amt) { ... } } // promise: withdrawal works
class FixedDeposit extends BankAccount {
    @Override public void withdraw(double amt) {
        throw new UnsupportedOperationException("Locked until maturity");
    }
}
void payRent(BankAccount acc) { acc.withdraw(15000); } // crashes for a FixedDeposit
```

payRent did nothing wrong. It trusted the contract; the subclass broke it.

The classic: Rectangle and Square

```
class Rectangle {
    protected int width, height;
    public void setWidth(int w) { width = w; }
    public void setHeight(int h) { height = h; }
    public int area() { return width * height; }
}
class Square extends Rectangle {
    @Override public void setWidth(int w) { width = w; height = w; }
    @Override public void setHeight(int h) { width = h; height = h; }
}

void resize(Rectangle r) {
    r.setWidth(5); r.setHeight(10);
    assert r.area() == 50; // a Square gives 100 – nothing threw, it just lied
}
```

The line the interviewer wants: "A square *is* a rectangle in geometry, but it is not **substitutable** in code, because callers rely on width and height being independent. Inheritance must follow **behaviour**, not the dictionary."

The four rules — a subclass must not...

Rule	Violation looks like
Demand more from callers (strengthen preconditions)	Parent accepts any positive amount; child rejects amounts over 2000
Deliver less than promised (weaken postconditions)	Parent's log() records the message; child's empty body records nothing
Break invariants the parent keeps	Rectangle's width/height independent; Square ties them
Throw new failures callers don't expect	UnsupportedOperationException from an inherited method

How to spot it in seconds

- An override that **throws** UnsupportedOperationException
- An override with an **empty body** that silently does nothing (often the worst — no error to notice)
- Callers writing if (x instanceof SomeSubclass) to dodge a subtype — **the giveaway**: if callers must check, substitution has already failed

The fix is the hierarchy, not the subclass. Make the parent promise only what **every** child can deliver, and move optional abilities into their own interfaces — which is Interface Segregation.

```
abstract class Account { public abstract double getBalance(); } // promises only what all can do
class SavingsAccount extends Account implements Withdrawable { ... }
class FixedDeposit extends Account { } // simply does not claim to be Withdrawable

void payRent(Withdrawable acc) { acc.withdraw(15000); } // compiler now prevents the old crash
```

Bonus point — Java violates LSP itself: Collections.unmodifiableList(x).add("y") throws UnsupportedOperationException. The type says List, which promises add(). Mentioning this shows you can spot the principle in the standard library — and that even good designers make the tradeoff knowingly.

The subtle distinction (you got this right in the check): LSP breaks when the PROMISE breaks, not when behaviour differs. EmailNotifier and SmsNotifier behave completely differently and both honour "the recipient is notified" — that's polymorphism working. What broke Liskov for Cash was refund() **throwing**, not cash refunds happening at a counter. A counter refund that issues a voucher honours the contract perfectly.

I — Interface Segregation Principle

Definition: no client should be forced to depend on methods it doesn't use. Prefer several small, focused interfaces over one fat one.

```
// VIOLATION
interface Worker { void work(); void eat(); void attendMeeting(); }

class Robot implements Worker {
    public void work() { ... }
    public void eat() { throw new UnsupportedOperationException(); }    // forced
    public void attendMeeting() { }                                    // meaningless
}
```

Look at HOW Robot copes: one method throws, one does nothing — the exact Liskov symptoms. Fat interfaces are one of the most common causes of LSP violations. The two principles are two sides of one problem: you spot an LSP violation, and you fix it with ISP.

```
// FIX — split by role
interface Workable { void work(); }
interface Feedable { void eat(); }
interface Collaborator { void attendMeeting(); }

class Human implements Workable, Feedable, Collaborator { ... }
class Robot implements Workable { ... }
```

Beyond avoiding exceptions: with a fat interface **every implementer is coupled to every method** — add `submitTimesheet()` to `Worker` and `Robot` must change too.

Counterweight: don't split into one-method interfaces everywhere. **Group methods clients use together.** Test from the client's side: does anyone need one without the other? If never, keep them together.

Exercise 13 — payment methods

What you got right: the interface split was correct, including **PayLater implementing only the EMI interface** — the awkward case designed to trap you. You also spotted the `instanceof` giveaway.

What you missed:

- ✗ You listed the violating methods but didn't **name which rule** each broke.
- ✗ Missed `Cash.pay()` 's 2000 limit — **demands more from the caller. It survived into your redesign:** splitting interfaces doesn't fix a precondition problem. Fix it by putting the limit *into the contract* (`boolean canPay(double amount)`), or a documented domain exception).
- ✗ Missed `Cash.saveForFuture()` 's empty body — silently does nothing.
- ✗ Didn't rewrite Checkout in Part B (task skipped again).
- ✗ Interface naming: interface `refund`, `payInEmi` — lowercase, named like methods.

The compile error that teaches types. After splitting, this stops compiling:

```
void processRefund(PaymentMethod pm, double amount) { pm.refund(amount); }    // ✗
```

The variable's DECLARED type decides which methods you can call, not the object inside it. `PaymentMethod` only has `pay()` now. Even `if (pm instanceof Refundable)` doesn't help — `pm` is still typed `PaymentMethod`, so you'd need a cast or pattern matching.

The better fix: ask for the capability in the signature.

```
void processRefund(Refundable method, double amount) { method.refund(amount); }
```

Now it is **impossible** to pass `Cash` or `PayLater` — the compiler rejects it before anything runs. A runtime crash became a compile error. That's the whole point of the redesign, and the answer to "does Checkout change when we add Wallet?" becomes **no**.

```
interface Payable { void pay(double amount); }
interface Refundable { void refund(double amount); }
interface EmiCapable { void payInEmi(double amount, int months); }
interface Savable { void saveForFuture(String userId); }

class CreditCard implements Payable, Refundable, EmiCapable, Savable { }
class Upi implements Payable, Refundable, Savable { }
class Cash implements Payable, Refundable { }    // counter refund honours the contract
class Wallet implements Payable, Refundable, Savable { }
class PayLater implements EmiCapable { }
```

Naming convention: interfaces are **capabilities** — capitalised, often -able (`Comparable`, `Runnable`, `Refundable`) — not lowercase verbs. `public` on interface methods is redundant.

D — Dependency Inversion Principle

Definition: high-level modules should not depend on low-level modules; both should depend on **abstractions**. Abstractions should not depend on details; details should depend on abstractions.

- **High-level** = business logic, the *policy*: "when an order is placed, charge the customer and notify them."
- **Low-level** = mechanics, the *details*: Razorpay's HTTP API, MySQL queries, the SMTP client.

Business rules are the valuable, stable part; details change constantly. Without DIP, your most important code depends on your most volatile code.

```
// VIOLATION – arrows point DOWN, into the details
class OrderService {
    private final RazorpayClient razorpay = new RazorpayClient();    // created inside → unswappable
    private final MySQLOrderDao dao = new MySQLOrderDao();
    public void placeOrder(Order o) { razorpay.charge(o.getAmount()); dao.insert(o); }
}

// FIX – both sides point AT the interface
interface PaymentGateway { void charge(double amount); }    // owned by the business side
interface OrderRepository { void save(Order o); }

class OrderService {
    private final PaymentGateway payment;
    private final OrderRepository repo;
    public OrderService(PaymentGateway payment, OrderRepository repo) {
        this.payment = payment; this.repo = repo;
    }
    public void placeOrder(Order o) { payment.charge(o.getAmount()); repo.save(o); }
}
class RazorpayGateway implements PaymentGateway { ... }    // detail depends on abstraction
class MySQLOrderRepository implements OrderRepository { ... }
```

That is the "inversion": the detail used to be depended *on*; now it does the depending.

The subtle part — who OWNS the interface

```
// ✗ interface mirroring the vendor – the detail leaked upward
interface PaymentGateway { RazorpayOrderResponse createRazorpayOrder(RazorpayRequest r); }

// ✓ interface expressing the business need
interface PaymentGateway { PaymentResult charge(String customerId, double amount); }
```

Test for a leaking abstraction: could you switch vendor **without touching the interface or its callers**? If any vendor type, response object, or vendor-shaped step appears, it has leaked.

DIP vs Dependency Injection — don't confuse them

Term	What it is
Dependency Inversion	A design principle : depend on abstractions, and let the high-level side own them
Dependency Injection	A technique : pass dependencies in (constructor/setter) instead of creating them inside

Injection is usually how you achieve inversion — but you can inject a concrete class (`new OrderService(new RazorpayClient())`) and still violate DIP. You need both: an interface type, **and** it passed in.

The payoff: testing — this is the answer to "how would you test this?"

```
class FakePaymentGateway implements PaymentGateway {
    boolean charged = false;
    public void charge(double amount) { charged = true; }
}

FakePaymentGateway fake = new FakePaymentGateway();
OrderService service = new OrderService(fake, new InMemoryOrderRepository());
service.placeOrder(order);
assertTrue(fake.charged);    // no network, no DB, milliseconds – and you can ASSERT behaviour
```

Your doubt: Spring, field vs constructor injection

"I didn't understand why Spring recommends constructor injection." Three separate arguments, unpacked:

```
// FIELD injection (older style)
@Service
public class OrderService {
    @Autowired
    private PaymentGateway payment;
    // no constructor at all
}

// CONSTRUCTOR injection (recommended)
@Service
public class OrderService {
    private final PaymentGateway payment;
    public OrderService(PaymentGateway p) {
        this.payment = p;
    }
}
```

Reason	Why
1. Testable without Spring (the big one)	<code>new OrderService()</code> leaves the fields null — private, no constructor, no setter, so nothing outside can fill them . Only Spring can (via reflection), so your unit test must boot the whole Spring context. With a constructor it's an ordinary Java class: <code>new OrderService(fake, fakeRepo)</code> .
2. Dependencies are explicit	The constructor signature tells you exactly what the class needs. With <code>@Autowired</code> fields you must scan the class body — and a class quietly growing to eight dependencies goes unnoticed. Eight constructor parameters look ugly, and that ugliness is useful feedback that SRP is broken .
3. Fields can be <code>final</code>	<code>final</code> = assigned exactly once, at construction; the object is valid the moment it exists. You cannot make an <code>@Autowired</code> field <code>final</code> , because field injection assigns <i>after</i> construction.

Every exercise you wrote from #5 onward used constructor injection. The constructor *is* the contract: "give me these collaborators and I'll work."

"What does reflection actually mean?"

Reflection lets code **inspect and modify other code at runtime** — look up classes, fields and methods **by name (as strings)** and call or change them, without knowing them at compile time.

```
// normal: compiler checks the name
e.getName();

// reflection: the name is DATA, resolved at runtime
Class<?> clazz = Class.forName("Employee");
Method m = clazz.getMethod("getName");
Object result = m.invoke(obj);           // typo "getNmæ" compiles fine, fails at runtime

// the part that matters for Spring – it can reach past private:
Field f = clazz.getDeclaredField("payment");
f.setAccessible(true);                   // ← switches OFF the encapsulation check
f.set(orderService, new RazorpayGateway());
```

That last block **is how @Autowired on a field works** — which is exactly why such a class can't be built in a test without Spring.

You've met reflection without knowing: JUnit finding `@Test` methods, Jackson/Gson reading fields to build JSON, Hibernate mapping `@Entity` fields to columns. Any time an annotation alone makes something happen, reflection (or bytecode generation) is underneath.

Downsides: no compile-time safety (names as strings), slower, breaks encapsulation, and IDE renames don't update the strings.

One-line answer if it comes up: "Frameworks use reflection to wire and configure objects from annotations. It's powerful but loses compile-time safety, so I prefer constructor injection, where the dependency is part of the type signature."

The Counterweights: DRY, KISS, YAGNI

Why this section exists. The five principles all push toward **more** abstraction. In interviews, over-engineering is marked down about as often as under-engineering — candidates who just learned patterns bolt three onto a problem that needed one.

DRY — Don't Repeat Yourself

Every piece of **knowledge** has one authoritative place. The key word is *knowledge*, not *code*. An 18% GST rate in three classes is a real violation — a law change means finding all three.

```
boolean isValidAge(int age)      { return age >= 18; }    // legal adulthood
boolean isValidSeatRow(int row) { return row >= 18; }    // cinema layout
```

Same shape, unrelated reasons. Merging these into `isAtLeast18()` couples two things that change independently — you'd be *creating* an SRP problem. **Duplication of knowledge is bad; coincidental similarity is fine.**

KISS — Keep It Simple

The simplest design meeting the requirements wins. A clear design that works beats a clever one with five patterns you can't finish in 45 minutes. **Signal:** can you explain each class's purpose in one sentence? If you must explain *why the abstraction exists* more than *what it does*, it's too clever.

YAGNI — You Aren't Gonna Need It

Don't build for requirements that don't exist. For "users pay by credit card", a `PaymentGatewayFactoryProvider` + `AbstractPaymentProcessorBase` + `PaymentStrategyRegistry` is three layers of speculation.

Resolving the tension with OCP

Build abstractions where change is known or likely. Leave seams where it's plausible. Don't build frameworks for the speculative.

Situation	Do
Requirement already lists multiple variants	Abstract now (Strategy etc.)
Interviewer hints "we might add X later"	Interface with one implementation — a cheap seam
Pure speculation, nobody mentioned it	Keep it concrete; mention the extension point verbally
It's an external system (payment, DB, email, maps)	Always put an interface in front — for testability alone. This row overrides YAGNI.

The line that scores well: "I've kept the borrow limit as a constant for now. If limits start varying by membership tier, I'd extract a `BorrowLimitPolicy` interface here — the seam is obvious, but with one fixed rule an interface would be indirection without benefit." Name the extension point, name where it goes, give a reason for declining. Interviewers often **want** to hear you decline an abstraction with a reason. Phrase it as a **decision**, not an apology.

Exercise 14 — what you got right and wrong

Part 1 (under-abstracted RideService):

- ✓ Named the OCP problem and the Demeter chain. `1c` was fully correct — fakes into the constructor.
- ✗ Missed the core DIP violation: `new GoogleMapsClient()` **created inside the class**. Even if it were an interface, `new` inside means nothing can ever be substituted — there is no injection point.
- ✗ **The vendor leaked into your interface:**

```
interface MapClient {
    public <T> getDistanceMatrix(String from, String to);    // x "distance matrix" is Google's shape
    public double getDistanceInKm(<T> distanceResponse);    // x <T> = "whatever the vendor returns"
}
```

The caller still held a response object and did a two-step dance that exists only because Google returns a matrix. **The <T> was the tell.** What does the business need? Not a matrix — just a number.

```
interface DistanceCalculator { double distanceInKm(String from, String to); }    // ✓ business language
interface RideNotifier      { void notifyFare(String phone, double fare); }

class GoogleDistanceCalculator implements DistanceCalculator {
    private final GoogleMapsClient client = new GoogleMapsClient();
    public double distanceInKm(String from, String to) {
        GoogleDistanceResponse r = client.getDistanceMatrix(from, to);
        return r.getRows().get(0).getElements().get(0).getDistanceKm();    // chain lives HERE
    }
}

class RideService {
    private final DistanceCalculator distance;
    private final RideNotifier notifier;
    public RideService(DistanceCalculator distance, RideNotifier notifier) { ... }
    public double bookRide(String riderPhone, String from, String to) {
        double km = distance.distanceInKm(from, to);
        double fare = km * 12;
        notifier.notifyFare(riderPhone, fare);
        return fare;
    }
}
```

The Demeter chain didn't disappear — it moved into the one class that is SUPPOSED to know Google's format. Same move as `customer.isGoldMember()` in exercise 11. Nothing in `RideService` now hints that Google or Twilio exist anywhere in the system. **That's the standard.**

Part 2 (over-abstracted library):

✓ Correctly cut the strategy machinery, and named YAGNI + KISS.

✗ **2b wrong:** you said keep `FixedBorrowLimitStrategy` — but that's part of the machinery you just removed. One fixed limit = `private static final int MAX_BORROW_LIMIT = 3;`

The thing to **keep** is `BookRepository` + `InMemoryBookRepository`, because it's an **external system boundary**: (1) you must substitute storage to test lending logic; (2) a real database is a *known* change, not speculation.

✗ 2c phrasing was hesitant ("we may allow more in future but that's not our scope") — sounds like an apology rather than a decision.

Carry this forward: abstract your boundaries (storage, payment, notification, external APIs); **keep business rules concrete** until they actually vary.

Interview Quick-Recall — Phase 2

Cover the right column and answer out loud in one or two sentences.

Question	Answer in one breath
State SRP precisely.	A class should have one reason to change — it should answer to one actor.
How do you test for an SRP violation?	Describe the class in one sentence without "and"; and list who would ask for it to change.
Is a class with 6 small tidy methods cohesive?	Not necessarily — cohesion is judged at class level. Six unrelated jobs is still low cohesion.
Does SRP mean one method per class?	No. Things that change together belong together; over-splitting is its own failure.
State OCP.	Open for extension, closed for modification — add behaviour with new code, don't edit working code.
"How would you add a new payment type?"	One new class implementing the interface plus one registration line; no existing file changes.
But something still has to pick the implementation?	Yes — that choice moves to one factory or map, instead of being scattered through business logic.
Fastest sign of an OCP violation?	if/else or switch on a type code, especially if repeated in several places.
State LSP.	A subtype must be usable anywhere the parent is expected without breaking correctness — it must keep the parent's promises.
Why isn't Square a Rectangle in code?	Callers rely on width and height being independent; Square breaks that invariant. Is-a in the real world ≠ substitutable in code.
Name the four ways to break LSP.	Demand more from callers, deliver less, break an invariant, throw unexpected failures.
Does different behaviour violate LSP?	No — only a broken promise does. SMS vs email both honour "the recipient is notified".
Quickest symptom of an LSP violation?	An override that throws UnsupportedOperationException or is empty; or callers using instanceof to dodge a subtype.
An example of LSP violation in the JDK?	Collections.unmodifiableList — the type promises add(), the implementation throws.
State ISP.	No client should be forced to depend on methods it doesn't use; prefer small role interfaces.
How are ISP and LSP related?	Fat interfaces force classes to fake methods — throwing or empty bodies — which is exactly how LSP breaks. Fix LSP with ISP.
Should every interface have one method?	No — group methods that clients use together. Split only when someone needs one without the other.
After splitting interfaces, how do you avoid instanceof?	Ask for the capability in the signature: processRefund(Refundable r). The compiler then enforces it.
State DIP.	High- and low-level modules both depend on abstractions; details depend on abstractions, not the reverse.
What exactly is "inverted"?	The dependency arrow — the detail used to be depended on; now it implements an interface the business side owns.
DIP vs DI?	DIP is the principle (depend on abstractions you own); DI is the technique (pass them in). You can inject a concrete class and still violate DIP.
How do you know an abstraction leaks?	If vendor types or vendor-shaped steps appear in the interface or its callers, you can't switch vendors without editing them.
"How would you unit test this service?"	Pass fake implementations into the constructor and assert on them — possible only because it depends on interfaces.
Constructor or field injection, and why?	Constructor: testable without the framework, dependencies explicit, fields can be final, and too many dependencies becomes visibly ugly.
What is reflection?	Inspecting and modifying classes at runtime using names as data; it can bypass private, which is how @Autowired fields get filled.
What does DRY actually forbid?	Duplicated knowledge, not duplicated-looking code. Coincidentally similar code should stay separate.
How do OCP and YAGNI coexist?	Abstract where change is known or likely; leave seams where plausible; keep speculation concrete. External systems always get an interface.
When would you deliberately NOT use a pattern?	When there's one variant today and no signal it'll grow — I'd name the extension point and keep it concrete.

Phase 2 Review Checklist

Run this over any design before calling it done.

- ☐ Can I describe every class in one sentence with no "and"?
- ☐ For each class, can I name the single actor who would request changes?
- ☐ Is there any if/else or switch on a type code? (→ polymorphism)
- ☐ If I add a new variant, how many existing files do I edit? (target: zero, or one registration line)
- ☐ Does any override throw UnsupportedOperationException or sit empty? (→ LSP; fix with ISP)
- ☐ Does any caller use instanceof to dodge a subtype? (→ ask for the capability type instead)
- ☐ Does any subtype add restrictions the contract never mentioned? (→ move the rule into the contract)
- ☐ Is any class forced to implement methods it can't support? (→ split the interface)
- ☐ Does any class create its own dependencies with `new`? (→ inject them)
- ☐ Do any vendor names or vendor types appear in my interfaces or business logic?
- ☐ Are all my fields typed as interfaces rather than concrete classes?
- ☐ Could I unit test each service by passing fakes into its constructor?
- ☐ Does every abstraction I built have a real, named reason to exist today? (→ else cut it)
- ☐ Do external systems (DB, payment, email, APIs) all sit behind an interface?

- ☐ Do all method signatures have proper return types and parameters? (your recurring gap — the signature IS the design)

Status: Phase 0 ✔ Phase 1 ✔ Phase 2 ✔ → **Next: Phase 3, Design Patterns.** You've already built **Strategy** three times, plus **Template Method** and **Adapter**, without naming them. Phase 3 puts names and structure on those, and adds the ones you haven't met: Singleton with thread safety, Factory, Builder, Observer, State, Decorator.